



Session 11. Sorting

1 Instructions

- This exercise must be implemented and submitted **as latest 24 hours before** your next laboratory class.
- Before submitting, rename the projects inside Eclipse as usual

`surname1_surname2_name_lab11`, in lowercase and without accents and

`surname1_surname2_name_util`.

Then, export them to `surname1_surname2_name_lab11_sorting` and upload it to the corresponding task in Campus Virtual.

2 Sorting

2.1 Java Collections class

It is used **exclusively** with static methods that operate on or return collections.

The important points about **Java Collections class** are:

- Java Collection class supports the polymorphic algorithms that operate on collections.
- Java Collection class throws a `NullPointerException` if the collections or class objects provided to them are null.

2.2 Java Collections `sort(...)` methods

Sort the elements in the specified list. Can be differentiated depending on the parameters.

- `sort(list)` method is used to sort the elements present in the specified list in ascending order.
- `sort(list, comp)` method is used to sort the specified list according to the order introduced by the specified comparator.

```
static <T extends Comparable<? super T>>  
void
```

```
sort(List<T> list)  
Sorts the specified list into ascending order, according to the natural  
ordering of its elements.
```

```
static <T> void
```

```
sort(List<T> list, Comparator<? super T> c)  
Sorts the specified list according to the order induced by the specified  
comparator.
```

2.3 To do

1. Download `lab11_sorting_initial`. There is a package `uo.mp.util.sorting` in source and test folder as well as class `Person` (in source folder) and `CollectionsTest` (in test folder).
2. Run the test. Uncomment `testStrings`. Run the test.



3. What if we have a list of custom objects? Of course, we should be able to sort them as well. Uncomment all the other tests and run.

2.4 Why doesn't work?

In order to sort this list, if we directly use the `Collections.sort(List list)`, it will give a **Compile Time Error** because there is no **natural ordering** defined for the `Person` objects. So, **it doesn't know how to sort this list**.

For objects to have a natural order, they must implement the interface `java.lang.Comparable`.

3 Using Comparable Interface

A **comparable object** is able of **comparing itself with another object**.

The class itself must implement the `java.lang.Comparable` interface to compare its instances. This interface imposes a **total ordering** on the objects of each class that implements it. This ordering is referred to as the **class's natural ordering** - the most common ordering of that object, and the class's `compareTo` method is referred to as its **natural comparison method**.

It is **strongly recommended** (though not required) that natural orderings be consistent with `equals`. So, for every `e1` and `e2` of class `C`

1. `e1.compareTo(e2) == 0` when `e1.equals(e2)`,
2. `e1.compareTo(e2) < 0` when `e1` comes first in the sequence than `e2`
3. `e1.compareTo(e2) > 0` when `e2` comes first in the sequence than `e1`

Let's define natural order for `Person` class as ascending by surname first then name, finally age. If we wish to sort a list of `Person` based on this natural ordering,

- `Person` must implement `Comparable` and
- override the method `compareTo()` using this criteria.

Do it!!!

Now, we want sort people by age first. When we make a collection element comparable (by having it implement `Comparable`), we get only one chance to implement the `compareTo()` method (the natural order). The solution is using `Comparator`.

4 Using Comparator

Unlike `Comparable`, `Comparator` is external to the element type we are comparing. It's a separate class. We create multiple separate classes (that implement `Comparator`) to compare by different members.

`Collections` class has a second `sort()` method and it takes `Comparator`. The `sort()` method invokes the `compare()` to sort objects.

To compare people by names first, then surname and finally age:



1. Create a class that implements Comparator (and thus the compare() method that does the work previously done by compareTo()). For example, create a class PersonComparator comparing by names first and then by surnames and ages.
2. Make an instance of the Comparator class.
3. Call the overloaded sort() method, giving it both the list and the instance of the class that implements Comparator.

Run the test.

5 Complete NewsStand project

5.1 Change options

Change the following options:

- 2.- Show publications will print the **publications sorted by descending sold first, descending stock and name.**
- 6.- Save orders to file will save the orders **sorted by number of copies (descending) and name**

Implement the classes and methods needed to sort the lists by these criterias